

A BOSS RUSH THAT LEARNS HOW YOU FIGHT

Rise, Tarnished

A browser based, single player boss duel game where every boss reads your recent play and refuses to let you win the same way twice.

Department of Computer Science and Engineering, North South University

Mustakim Ahmed Hasan · Md Rafiul Haque · Md Mafiul Haque

Project status presentation, prepared from the live repository and sprint records



Boss fights are the best part of these games, and the hardest to reach

Soulslike games sell players on tense, one on one duels against memorable bosses. In practice a player has to grind through hours of traversal, side content, and leveling before they ever reach the fight they actually wanted. Once a boss is beaten a few times, its attack pattern becomes predictable and repeat attempts stop feeling dangerous, they start feeling scripted. Mastery turns into memorization instead of a real skill test, and there is no quick, browser based way to get that duel feeling without installing a large game and committing hours to it.

WHAT WE SET OUT TO BUILD

- ✓ A short, browser based session built around one boss duel at a time, no traversal and no grinding to reach the fight
- ✓ A boss whose fighting style reacts to the specific player in real time, so a memorized dodge pattern stops working on its own
- ✓ Attack sequencing that is reweighted between attempts, so no two runs against the same boss play out identically
- ✓ Enough fairness built into that adaptation that the fight always stays winnable, never becomes an unbeatable wall

What the system has to do, and what it has to survive



Functional requirements

- Let a player sign in with a Google account and keep their save data across sessions
- Give the player movement, a light attack, a heavy attack, a dodge roll with invincibility frames, a block, and a ranged sorcery cast
- Gate those actions behind a stamina pool and a separate focus point pool, so a player cannot act without limit
- Track poise and posture so enough incoming damage interrupts or staggers either fighter
- Read the player behavior over the last stretch of the fight and shift the boss tactics and move choices in response
- Resolve a win or a loss, pay out currency called runes, and unlock the next region on a kill
- Let the player spend runes on vitality, dexterity, and intelligence, and have each stat measurably change combat
- Store every player stats, progress, and attempt history in a persistent database
- Produce a short written explanation of exactly what killed the player after every death



Non functional requirements

- Combat has to hold sixty frames per second on a mid tier laptop, which rules out calling an AI model from inside the fight loop
- The adaptive boss brain must be a real time deterministic system, the same inputs and seed always produce the same fight
- One player data must be completely isolated from every other player, enforced at the database level, not only in app code
- Core game logic, the combat math and the boss brain, must reach at least eighty percent automated test coverage
- That same logic must be testable without opening a browser, since it is what the team has to trust the most
- The game must stay fully playable if the AI recap service is slow or unreachable, it can never block a fight
- The codebase has to stay small and clear enough for the team to add new bosses without breaking old ones

Why this is buildable by the team, on a real schedule



Technical

Every layer of the stack, Next.js, Phaser, Supabase, is mature and well documented, so there is no research risk in the tooling itself

The hardest idea, a boss that adapts to the player, is solved with a plain three layer state machine driven by measured signals, not a trained model, so it runs every frame with zero latency

That determinism is proven, not assumed, using property based tests that throw thousands of generated adversarial play patterns at the boss brain



Operational

The project runs on one week sprints with a single written goal each week, and six sprints have already shipped real, merged software

Every change goes through its own branch, an automated pipeline, and a self review pass before it reaches the main branch

Continuous integration spins up a real disposable database for every change, to prove data isolation actually holds, not just that it compiles



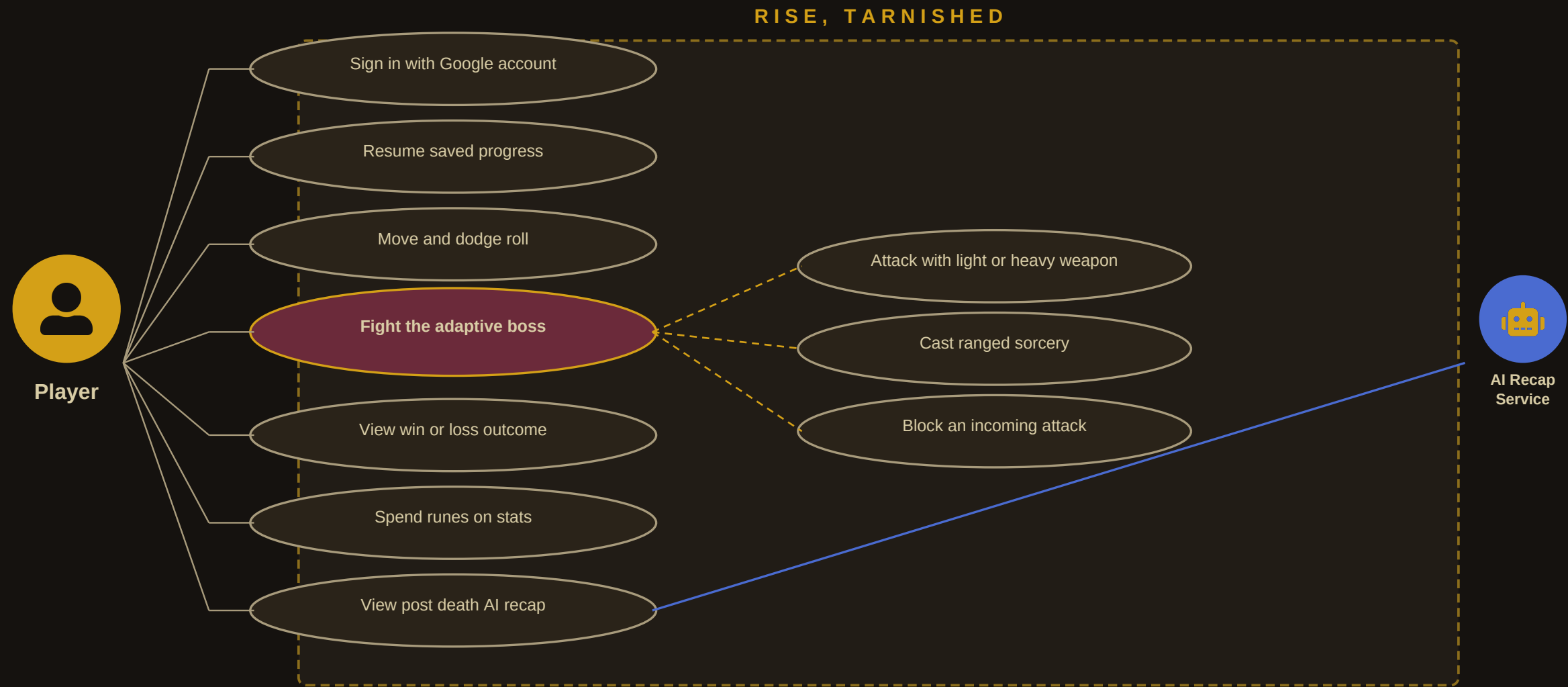
Economic

Hosting on Vercel and the database and auth on Supabase are both free at the project current scale

The AI recap call only runs once per death, not every frame, so the AI provider bill stays small and pay as you go

No paid game engine license and no paid art tools were needed to reach the current state, keeping build cost near zero

Use case diagram of the game system



Dashed gold lines are an include relationship: the three ways to fight are part of the fight use case, not separate goals of their own. The AI recap runs after death, outside the real time loop.
RISE, TARNISHED / PROJECT STATUS PRESENTATION

What the game is actually built with



Frontend and app shell

Next.js with the App Router, React, and TypeScript in strict mode for the menus, sign in flow, and the results screen



Game engine

Phaser, mounted inside a single Next.js client component with a StrictMode safe lifecycle, handles rendering, input, and hitboxes only



Real time boss AI

A hand written three layer hierarchical state machine in plain TypeScript, phase then tactic then action, with a seeded random source so every fight is reproducible



Authentication and data

Supabase for Google OAuth sign in and a Postgres database, with Row Level Security enforcing that a player can only ever touch their own rows



Asynchronous AI

The OpenAI API, called only from a server route after a death, never from the browser and never inside the combat loop



Testing

Vitest for unit tests and fast check for property based fairness testing against thousands of simulated adversarial fights

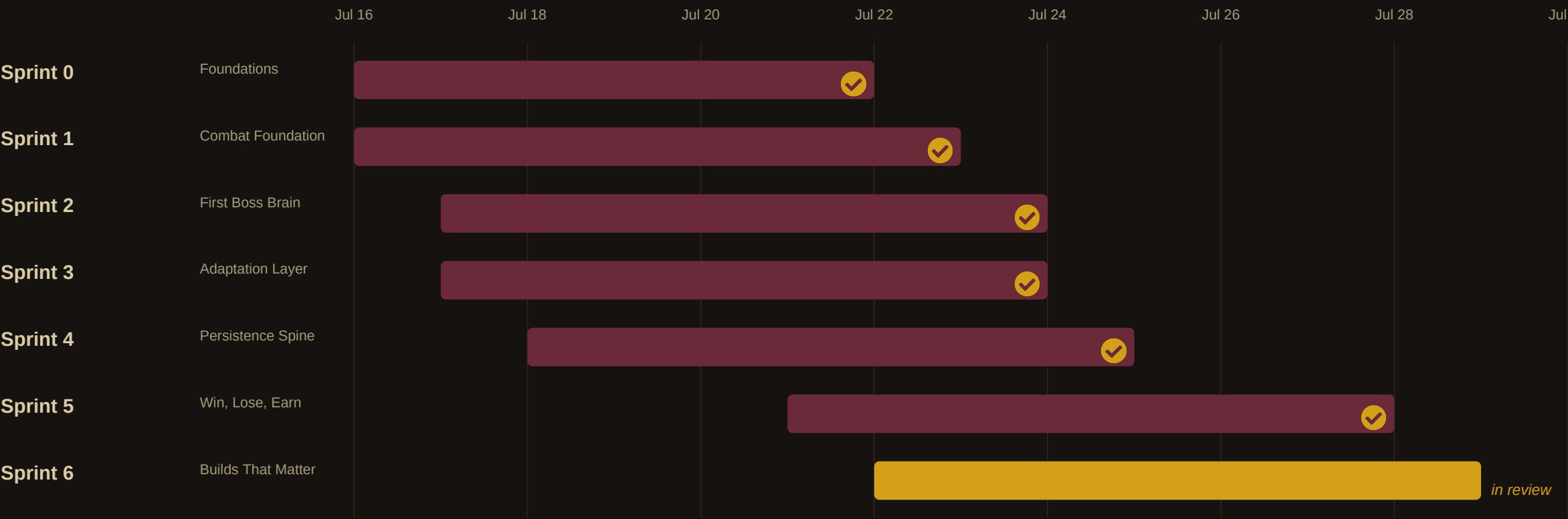


Process and hosting

GitHub with branch protection and a Projects board, GitHub Actions for continuous integration, Vercel for the app, Supabase for the database

PROJECT TIMELINE

Six shipped sprints, one week each, one goal each



WHAT COMES NEXT

Finish stat spending and its server side validation, then the post death AI recap, then the first full art pass with real character sprites and a real background in place of placeholder shapes.

What already exists and runs today

12

issues shipped
and merged

140

automated
unit tests

19

automated database
security tests

6

one week sprints
completed

- ✓ A complete real time combat system, movement, stamina, four melee actions, poise, and posture, written as pure logic with no game engine dependency so every rule is unit tested on its own
- ✓ A working three layer adaptive boss brain for the first boss, tracking seven behavior signals in a rolling window and reweighting its own move choices based on how the player has actually been fighting
- ✓ That adaptive brain fairness proven with property based tests, over sixty thousand simulated adversarial decisions across five bot playstyles, zero fairness rule violations found
- ✓ Google sign in working end to end, with a Postgres schema and Row Level Security so every player save data is provably isolated from every other player
- ✓ A complete win and loss loop, a fight now ends for real, pays runes, and unlocks the next region, all validated on the server so the client can never fake a reward
- ✓ A new focus point resource and a ranged cast action for the intelligence build, giving that stat archetype a real attack for the first time, currently in code review

CURRENT STATUS

The core loop is real. Sign in, fight, win or lose, earn, spend.

What remains for the first playable slice is finishing stat spending, adding the post death AI recap, and a real art pass. All work is tracked issue by issue on a public board and shipped through reviewed pull requests.

